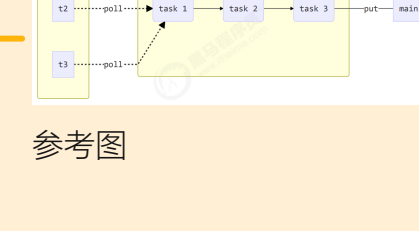


存在意义

- 减少开销
 - 在大部分JVM上,用户线程与操作系统的线程1:1的关系,也就是每创建一个线程和回收线程都要进行内核调用,开销较大。那么有了线程池就可以重复使用线程资源,大幅降低创建和销毁的频率。
- 便于管理
 - 线程池可以维护线程ID,线程状态等信息,也可帮你统计任务执行状态等信息。
- 提高响应速度
 - 当任务到达时,任务可以不需要等到线程创建就能立即执行。



线程池数量是有限的,当所有线程都在处理任务的时候我们需要一个队列来存放任务。

任务队列类如何设计?

- 1.任务队列当然需要任务队列了
- 2.锁
 - 当我们在开发环境下,线程要获取任务的时候需要用到锁,否则万一一个任务被三个线程执行不乱套了
- 3.生产者条件变量
 - 当生产条件生产的任务把队列占满了我们考虑让生产者阻塞
- 4.消费者条件变量
 - 同理消费者没有任务要执行的时候也考虑阻塞
- 5.容量
 - 任务队列肯定需要是一个最大容量的

属性设置?

任务队列的实现?

```
public class TaskQueue {
    private final int capacity;
    private final Task[] tasks;
    private final AtomicInteger count = new AtomicInteger(0);
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();

    public TaskQueue(int capacity) {
        this.capacity = capacity;
        this.tasks = new Task[capacity];
    }

    public void add(Task task) {
        lock.lock();
        try {
            while (count.get() == capacity) {
                notFull.await();
            }
            tasks[count.get()] = task;
            count.incrementAndGet();
            notEmpty.signal();
        } finally {
            lock.unlock();
        }
    }

    public Task remove() {
        lock.lock();
        try {
            while (count.get() == 0) {
                notEmpty.await();
            }
            Task task = tasks[count.get() - 1];
            tasks[count.get() - 1] = null;
            count.decrementAndGet();
            notFull.signal();
            return task;
        } finally {
            lock.unlock();
        }
    }
}
```

阻塞获取

阻塞添加

带超时阻塞获取

带超时阻塞添加

大小

拒绝策略的方法尝试put

这边主要是看拒绝策略是否用这个方法

自定义线程池

如果我们自己设计一个小的线程池,该怎么设计? 面试题!

- 2.线程集合
 - 执行的线程数量是有限的,所以设置一个线程集合
- 3.核心线程数
 - 也就是普通状态下最大的常驻线程数量,不然没得工作
- 4.获取任务时候的超时时间
 - 设置这个主要是如果长时间线程没有获取到任务,那么我们就考虑回收该线程,然后也可没有这个设置,那就是不回收策略
- 5.拒绝策略接口
 - 当任务队列也满了的时候,我们需要考虑新来的任务拒绝策略

- 1.丢弃
- 2.带超时等待
- 3.放弃任务执行
- 4.抛出异常
- 5.让调用者自己执行任务

设计

任务队列属性一推

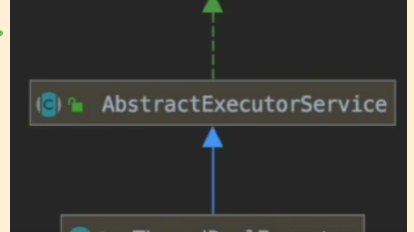
这边Worker类是内部类继承Thread,可以不断的获取任务,实现如图

这可以发现构造方法中有一个rejectPolicy是实现未实现的接口,将我们传入的具体操作来实现!

方法如何实现? 方法主要就是实现一个执行任务

测试

拒绝策略真的是根据我们自己的想法实现就实现吧



静态变量

ThreadLocalExecutor使用int的最低3位来表示线程池状态,低29位表示线程数

COUNT_BITS 用于进行移位操作

CAPACITY 表示了线程池三位为0,其他位都是1,代表了线程池所能维护的最大线程数。

ctl 通过原子性操作ctl这个值来维护状态。

workQueue

JDK提供可选的任务队列

ArrayBlockingQueue 是一个基于数组结构的有界阻塞队列,此队列按FIFO(先入先出)顺序对元素进行排序。

LinkedBlockingQueue 一个基于链表的阻塞阻塞队列,此队列按FIFO排序元素,吞吐量通常要高于ArrayBlockingQueue。静态工厂方法Executors.newFixedThreadPool()作用了这个队列。

SynchronousQueue 一个不存储元素的阻塞队列。每个插入操作必须等到另一个线程调用移除操作,否则插入操作一直处于阻塞状态。吞吐量通常要高于LinkedBlockingQueue。静态工厂方法Executors.newCachedThreadPool()作用了这个队列。

PriorityBlockingQueue 一个具有优先级的无限阻塞队列

mainLock 对线程池的一些操作需要状态同步,会用到锁

workers 这些是Worker,这个内部类可以视为对工作线程以及一些状态的管理,workers是存储他的集合

termination mainLock创建的Condition,应该是terminal调用时的线程阻塞

largestPoolSize 线程池中最多有过多少个活跃线程

completedTaskCount 线程池总共处理了多少任务

threadFactory 类型为ThreadFactory的接口,用户可以通过该接口来创建线程的工厂。可以通过线程工厂给每个创建出来的线程设置名字,使用类名ThreadFactoryBuilder可以快速的给线程池里的线程设置有意义的名字

handler 拒绝策略,当workQueue满就触发

JDK1.5中提供的四种选择

AbortPolicy 直接抛出异常。

CallerRunsPolicy 只用调用者所在线程来运行任务。

DiscardOldestPolicy 丢弃队列里最近的一个任务,并执行当前任务。

DiscardPolicy 不处理,丢弃掉。

keepAliveTime 工作线程空闲时保持存活的时间

allowCoreThreadTimeOut 是否需要保持核心线程始终处于存活

corePoolSize 核心线程数,可以看作待命的工作线程数量,当线程数达到饱和状态时,超过这个数量的线程将被终止

maximumPoolSize 最大线程数,可以看作待命的工作线程数量,当线程数达到饱和状态时,超过这个数量的线程将被终止

ThreadPoolExecutor(JDK实现的)

构造方法

Executors 类中提供了众多工厂方法来创建各种用途的线程池

newFixedThreadPool

特点

评价

newCachedThreadPool

特点

评价

newSingleThreadExecutor

使用场景

区别

子主题 4

说明Worker内部存在同步需求

继承AQS

Worker内部有什么需要同步的状态? 其实要回想一下线程池的状态

实现Runnable

Worker本身就是一个异步的任务调查者

构造方法把state初始化为-1,不能初始化为0

是为了在初始化期间不能接收中断信号,直到runWorker方法开始运行, state才会修改为0,并由于线程被阻塞,所以runWorker可以接收当前线程的中断信号了。

内部类

最重要的runWorker方法也就是run方法

getTask

预期

对源码的说明

执行流程图

execute

解释

深入进阶addWorker

解释说明

父类中的提交

shutdown

shutdownNow

其他方法

创建多少线程池合适?

过少会导致程序不能充分地利用系统资源,容易造成瓶颈;过大会导致更多的线程上下切换,占用更多内存

1.CPU密集型运算

通常采用 cpu 核数 + 1 能够实现最佳的CPU 利用率。+1 是保证当线程由于系统负载(网络堵塞)或访问慢(数据库慢)时,额外的这个线程能顶上去,保证CPU 时间不被浪费

2.IO密集型计算

CPU 不会处于繁忙状态。例如,当你执行业务逻辑时,这时就会使用 CPU 资源,但是当你的执行 I/O 操作时,进程 RPC 调用时,包括进行数据库操作时,这时被 CPU 闲置下来了,你可以利用多线程提高CPU 利用率。

经验公式: 线程数 = 核数 * 期望 CPU 利用率 * 总时间(CPU 计算时间 + 等待时间) / CPU 计算时间